Autonomous Emergency Braking System (AEBS)

System Specification and Formal Abstraction

Heba Al Kayed

Abstract

This document formally specifies the dynamics of the AEBS case study as implemented in the Python simulation environment. The system is modelled as a **Discrete-Time Stochastic Hybrid System (dt-SHS)**, obtained via Euler discretization of standard continuous benchmarks. It details the physics model, sensor noise characteristics, and the dual-mode controller logic. Finally, it describes the abstraction pipeline used to generate the Interval Markov Decision Process (IMDP) for formal verification, adapting finite abstraction techniques from [1].

1 System Architecture

The system is modelled as a closed-loop simulation consisting of four distinct components:

- **Vehicle Plant ($\mathcal{P}$):** A discrete-time physical model of the ego vehicle.
- **Traffic Environment ($\mathcal{E}$):** Manages the dynamics of the lead vehicle and scenario configuration.
- **Perception System ($\mathcal{S}$):** Acts as a stochastic observer function $\mathbf{y} = f(\mathbf{x})$.
- **Controller ($\mathcal{C}$):** A deterministic reactive policy mapping observations to control inputs.

2 Vehicle Plant Dynamics

The vehicle dynamics are implemented in the `VehiclePlant` class. The system state evolves in discrete time steps of $\Delta t = 0.1s$.

2.1 State Space Definition

The state vector is defined as $\mathbf{x}_k = [x_k, v_k, a_k]^T$, where:

- x_k: Position (or Gap) at step k.
- v_k: Velocity (or Closing Speed) at step k.
- a_k: Longitudinal Acceleration at step k.

The system supports two coordinate references:

- **World Frame:** x denotes absolute position on the track (increasing).
- **Relative Frame:** x denotes the **Gap** to the obstacle (decreasing). This frame is utilized for formal abstraction.

2.2 Discrete-Time Update Law

The physics engine implements a **Semi-Implicit Euler** update scheme. Unlike basic Euler integration, this method uses the updated acceleration to compute velocity, and the *updated* velocity to compute position.

This distinction is crucial for simulation fidelity. Semi-Implicit Euler is symplectic, meaning it better preserves the physical structure of the system's phase space. For mechanical systems, it tends to dampen artificial energy growth, behaving more like the true continuous flow than standard explicit methods.

The state update equations for the transition $t_k \to t_{k+1}$ are:

$$a_{k+1} = a_k + \alpha \cdot (u_k - a_k) \tag{1}$$

$$v_{k+1} = \max(0, v_k + a_{k+1} \cdot \Delta t) \tag{2}$$

$$x_{k+1} = \begin{cases} x_k + v_{k+1} \cdot \Delta t & \text{if world\_frame} \\ x_k - v_{k+1} \cdot \Delta t & \text{if relative\_frame} \end{cases} \tag{3}$$

Where:

- u_k: The control input (acceleration command) at step k.
- Δt: The time step duration ($0.1s$).
- α: The actuator lag coefficient ($\alpha = 0.5$), representing the delay in reaching commanded acceleration.

```python
# 1. Update Acceleration (First-Order Lag)
# Maps to Eq (1): a_new = a + alpha * (u - a)
self.actual_acceleration = self.actual_acceleration + self.alpha * (action_acceleration
    - self.actual_acceleration)

# 2. Update Velocity
# Maps to Eq (2): v_new = v + a_new * dt
self.actual_velocity += self.actual_acceleration * self.dt

# Constraints: No negative velocity
if self.actual_velocity < 0.0:
    self.actual_velocity = 0.0
    self.actual_acceleration = 0.0

# 3. Update Position
# Maps to Eq (3): x_new = x + v_new * dt
if self.coordinate_system == 'world_frame':
    self.x += self.actual_velocity * self.dt
else:
    # Relative Frame: x is GAP. Positive closing speed REDUCES the gap.
    self.x -= self.actual_velocity * self.dt
```

Listing 1: Implementation of Semi-Implicit Euler Step (from `vehicle_plant.py`)

3 Perception and Sensor Noise Model

The `PerceptionSystem` implements the observation function. It introduces stochasticity via two mechanisms: aleatoric measurement noise (Gaussian jitter) and probabilistic detection failure (Bernoulli dropouts).

3.1 Observation Function

The observed state $\mathbf{y}_k = [\tilde{g}, \tilde{v}]$ is derived from the ground truth $\mathbf{x}_k$:

$$\mathbf{y}_k = \begin{cases} \emptyset(\text{False}) & \text{with probability } P_{FN} \\ \mathbf{x}_k + \mathbf{w}_k & \text{otherwise} \end{cases} \tag{4}$$

Where:

- P_{FN}: The False Negative rate (probability the sensor misses the car entirely).
- $\mathbf{w}_k$: The noise vector drawn from independent Gaussian distributions.

$$\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{\Sigma}), \quad \mathbf{\Sigma} = \begin{bmatrix} \sigma_{pos}^2 & 0 \\ 0 & \sigma_{vel}^2 \end{bmatrix}$$

```python
# 1. Detection Logic (Classification DNN)
# Maps to P_FN logic
if np.random.rand() < self.fn_rate:
    return False, real_gap, real_v_rel

# 2. Estimation Logic (Regression DNN)
# Maps to w_k ~ N(0, Sigma)
gap_noise = np.random.normal(0, self.pos_noise)
vel_noise = np.random.normal(0, self.vel_noise)

obs_gap = max(0.0, real_gap + gap_noise)
obs_v_rel = real_v_rel + vel_noise
```

Listing 2: Sensor Noise Injection (from `perception.py`)

4 Controller

The controller implements a reactive decision logic. It does not access the ground truth state $\mathbf{x}_k$; instead, it relies entirely on the noisy observation $\mathbf{y}_k$ and its own internal belief of its velocity state.

To support comprehensive benchmarking, the system implements two distinct parameter sets representing different design philosophies.

4.1 Control Logic and Benchmarks

The control logic calculates Time-To-Collision ($TTC = \text{gap}/v_{rel}$) based on the observed inputs and compares it against pre-defined thresholds.

- **Industry Mode:** Represents a standard consumer vehicle setup. It prioritizes passenger comfort and traffic flow, engaging braking later and more aggressively only when necessary.
- **Safe Mode:** Represents a conservative safety-critical setup (e.g., autonomous shuttle). It prioritizes collision avoidance above all else, engaging braking earlier.

Table 1: Controller Benchmark Parameters (Comparison)

Parameter	**Symbol**	**Industry Mode**	**Safe Mode**
Time Thresholds			
Warning TTC	TTC_{warn}	2.6s	6.0s
Braking TTC	TTC_{brake}	1.6s	5.0s
Emergency TTC	TTC_{crit}	1.0s	4.0s
Distance Thresholds			
Warning Dist	d_{warn}	10.0m	15.0m
Braking Dist	d_{brake}	6.0m	10.0m
Emergency Dist	d_{crit}	2.0m	5.0m
Actuation			
Brake Force	u_{brake}	-4.0 m/s^2	-4.0 m/s^2
Emergency Force	u_{crit}	-9.8 m/s^2	-8.0 m/s^2

4.2 Decision Logic Implementation

The logic branches based on observed closing velocity ($v_{rel} < 5.0m/s$) to distinguish between low-speed approach (using distance thresholds) and high-speed collision avoidance (using TTC thresholds).

```python
# 1. Update Internal Belief (State Estimation)
# The controller tracks its own estimated velocity, separate from ground truth
self.est_velocity = max(0.0, prediction + noise)

# 2. Decision Logic (Using Observed Gap/Velocity)
if obs_v_rel < 5.0:
```

```
    if obs_gap < self.dist_emergency: req_acc, self.state = self.acc_emergency, 'emergency_brake'
    elif obs_gap < self.dist_brake:   req_acc, self.state = self.acc_brake, 'brake'
else:
    if ttc < self.ttc_emergency:      req_acc, self.state = self.acc_emergency, 'emergency_brake'
    elif ttc < self.ttc_brake:        req_acc, self.state = self.acc_brake, 'brake'

return req_acc, self.state
```

Listing 3: Reactive Decision Tree (from `controller.py`)

5 Formal Abstraction Methodology

The formal verification of the AEBS relies on constructing a finite abstraction of the continuous dynamics. Our methodology is based on the framework for finite abstractions of Stochastic Hybrid Systems (SHS) as surveyed in [1] (specifically Section 5), but we extend the standard approach to generate an **Interval Markov Decision Process (IMDP)** rather than a standard MDP.

5.1 Algorithm Adaptation: Why IMDP?

Standard abstraction algorithms typically partition the state space into a grid and compute transition probabilities $P(s'|s,a)$ by integrating the stochastic kernel over target cells. However, this approach yields a precise probability distribution that assumes perfect abstraction, ignoring the discretization error inherent in gridding a continuous space.

To represent this uncertainty rigorously, we compute the discretization error ϵ derived from the system's Lipschitz constant L and the grid resolution δ:

$$\epsilon = L \cdot \frac{\delta_{max}}{2}$$

Instead of a single probability p, we generate probability intervals $[p-\epsilon, p+\epsilon]$. This necessitates the use of an IMDP, which allows us to bound the behaviour of the continuous system within these intervals, ensuring that the verification results remain sound even with a coarse grid.

5.2 The Abstraction Pipeline

The Python implementation follows a modular three-stage pipeline:

1. **System Wrappers:** The concrete `VehiclePlant` and `AEBSController` classes are wrapped in a generic `StochasticHybridSystem` interface. This exposes the continuous dynamics (via the `get_next_state_distribution` method) and action spaces to the discretizer.

2. **Kernel Integration (Discretizer):** For every state cell center c in the grid $\mathcal{G}$ and every action a:

   - The next expected state μ and process noise σ are queried from the plant wrapper.
   - The transition probability to every potential target cell T is computed via Gaussian integration:
   $$p_{target} = \int_T \mathcal{N}(x; \mu, \sigma)\, dx$$
   - The bounds are expanded by the error term ϵ to form the IMDP transition:
   $$P(s \to s') \in [p_{target} - \epsilon, p_{target} + \epsilon]$$

3. **Compositional Export:** The resulting intervals are exported to the PRISM language. The final model is a parallel composition of the `Plant`, `Perception`, and `Controller` modules.

5.3 Synchronization Protocol

To faithfully reproduce the sequential nature of the closed-loop simulation within the PRISM model, a specific synchronization protocol is enforced. This prevents race conditions and ensures that the controller acts only on fresh observations.

The modules synchronize via a shared global variable $t \in \{1, 2, 3\}$, which cycles through the system components in a strict order:

1. **Physics Step ($t = 1$):** The `VehiclePlant` evolves the physical state ($s \to s'$).
2. **Perception Step ($t = 2$):** The `PerceptionSystem` updates the observed variables ($y \leftarrow s'$).
3. **Control Step ($t = 3$):** The `AEBSController` reads y and updates the action variable ($u \leftarrow \pi(y)$).

This protocol ensures that within one discrete time step Δt, the causal chain $State \to Observation \to Action$ is preserved.

References

[1] A. Lavaei, S. Soudjani, A. Abate, and Z. Majid. Automated verification and synthesis of stochastic hybrid systems: A survey. *arXiv preprint arXiv:2203.02476*, 2022. See Section 5: Finite Abstractions.